

Introduction to Machine Learning and Well-Posed Learning Problems

December 9, 2025

1.2 Designing a Learning System

- We illustrate basic design issues in machine learning

1.2 Designing a Learning System

- We illustrate basic design issues in machine learning
- Example: design a program to **learn to play checkers**

1.2 Designing a Learning System

- We illustrate basic design issues in machine learning
- Example: design a program to **learn to play checkers**
- Ambitious goal:
 - Enter it in the **world checkers tournament**
 - Use an “obvious” performance measure

1.2 Designing a Learning System

- We illustrate basic design issues in machine learning
- Example: design a program to **learn to play checkers**
- Ambitious goal:
 - Enter it in the **world checkers tournament**
 - Use an “obvious” performance measure
- Performance measure:
 - **Percent of games it wins in the world tournament**

Recall: Well-Posed Learning Problem

- A learning problem is specified by:
 - **Task** T

Recall: Well-Posed Learning Problem

- A learning problem is specified by:
 - **Task** T
 - **Performance measure** P

Recall: Well-Posed Learning Problem

- A learning problem is specified by:
 - **Task** T
 - **Performance measure** P
 - **Training experience** E

Recall: Well-Posed Learning Problem

- A learning problem is specified by:
 - **Task T**
 - **Performance measure P**
 - **Training experience E**
- For checkers:
 - Task T : Playing checkers

Recall: Well-Posed Learning Problem

- A learning problem is specified by:
 - **Task T**
 - **Performance measure P**
 - **Training experience E**
- For checkers:
 - Task T : Playing checkers
 - Performance measure P : % of games won in the world tournament

Recall: Well-Posed Learning Problem

- A learning problem is specified by:
 - **Task T**
 - **Performance measure P**
 - **Training experience E**
- For checkers:
 - Task T : Playing checkers
 - Performance measure P : % of games won in the world tournament
 - Training experience E : *to be chosen*

Recall: Well-Posed Learning Problem

- A learning problem is specified by:
 - **Task T**
 - **Performance measure P**
 - **Training experience E**
- For checkers:
 - Task T : Playing checkers
 - Performance measure P : % of games won in the world tournament
 - Training experience E : *to be chosen*
- Section 1.2 focuses now on:
 - **How to choose the training experience E**

1.2.1 Choosing the Training Experience

- First design choice: **type of training experience**

1.2.1 Choosing the Training Experience

- First design choice: **type of training experience**
- The type of training experience can greatly affect:
 - Success or failure of the learner

1.2.1 Choosing the Training Experience

- First design choice: **type of training experience**
- The type of training experience can greatly affect:
 - Success or failure of the learner
- Three important attributes of the training experience:
 - 1 Whether feedback is **direct or indirect**

1.2.1 Choosing the Training Experience

- First design choice: **type of training experience**
- The type of training experience can greatly affect:
 - Success or failure of the learner
- Three important attributes of the training experience:
 - ① Whether feedback is **direct or indirect**
 - ② How much the **learner controls** the training examples

1.2.1 Choosing the Training Experience

- First design choice: **type of training experience**
- The type of training experience can greatly affect:
 - Success or failure of the learner
- Three important attributes of the training experience:
 - ① Whether feedback is **direct or indirect**
 - ② How much the **learner controls** the training examples
 - ③ How well training data **matches the test distribution**

Attribute 1: Direct vs Indirect Feedback

- Key question:
 - Does the training experience give **direct feedback** on each choice?

Attribute 1: Direct vs Indirect Feedback

- Key question:
 - Does the training experience give **direct feedback** on each choice?
 - Or only **indirect feedback** via the final outcome?

Attribute 1: Direct vs Indirect Feedback

- Key question:
 - Does the training experience give **direct feedback** on each choice?
 - Or only **indirect feedback** via the final outcome?
- **Direct feedback** example in checkers:
 - Training examples: individual board states + correct move

Attribute 1: Direct vs Indirect Feedback

- Key question:
 - Does the training experience give **direct feedback** on each choice?
 - Or only **indirect feedback** via the final outcome?
- **Direct feedback** example in checkers:
 - Training examples: individual board states + correct move
 - Learner is told explicitly which move is best in each position

Attribute 1: Direct vs Indirect Feedback

- Key question:
 - Does the training experience give **direct feedback** on each choice?
 - Or only **indirect feedback** via the final outcome?
- **Direct feedback** example in checkers:
 - Training examples: individual board states + correct move
 - Learner is told explicitly which move is best in each position
- **Indirect feedback** example in checkers:
 - Training experience: only full games and final outcome (win/loss)

Attribute 1: Direct vs Indirect Feedback

- Key question:
 - Does the training experience give **direct feedback** on each choice?
 - Or only **indirect feedback** via the final outcome?
- **Direct feedback** example in checkers:
 - Training examples: individual board states + correct move
 - Learner is told explicitly which move is best in each position
- **Indirect feedback** example in checkers:
 - Training experience: only full games and final outcome (win/loss)
 - No explicit label for each individual move

Credit Assignment Problem

- With indirect feedback, the learner faces **credit assignment**:
 - How much credit or blame does each move deserve for the final outcome?

Credit Assignment Problem

- With indirect feedback, the learner faces **credit assignment**:
 - How much credit or blame does each move deserve for the final outcome?
- Difficult because:
 - The game can be lost even if **early moves are optimal**

Credit Assignment Problem

- With indirect feedback, the learner faces **credit assignment**:
 - How much credit or blame does each move deserve for the final outcome?
- Difficult because:
 - The game can be lost even if **early moves are optimal**
 - A few poor moves later may cause the loss

Credit Assignment Problem

- With indirect feedback, the learner faces **credit assignment**:
 - How much credit or blame does each move deserve for the final outcome?
- Difficult because:
 - The game can be lost even if **early moves are optimal**
 - A few poor moves later may cause the loss
- Therefore:
 - **Learning from direct training feedback** is typically easier

Credit Assignment Problem

- With indirect feedback, the learner faces **credit assignment**:
 - How much credit or blame does each move deserve for the final outcome?
- Difficult because:
 - The game can be lost even if **early moves are optimal**
 - A few poor moves later may cause the loss
- Therefore:
 - **Learning from direct training feedback** is typically easier
 - **Learning from indirect feedback** is more challenging

Attribute 2: Who Controls the Training Examples?

- Second attribute: **degree of learner control** over training examples

Attribute 2: Who Controls the Training Examples?

- Second attribute: **degree of learner control** over training examples
- Possible settings:
 - 1 Teacher-selected examples

Attribute 2: Who Controls the Training Examples?

- Second attribute: **degree of learner control** over training examples
- Possible settings:
 - ① Teacher-selected examples
 - ② Learner queries the teacher

Attribute 2: Who Controls the Training Examples?

- Second attribute: **degree of learner control** over training examples
- Possible settings:
 - ① Teacher-selected examples
 - ② Learner queries the teacher
 - ③ Learner generates its own experience

Teacher-Selected vs Learner-Selected Examples

- **Teacher-selected** examples:
 - Teacher chooses informative board states

Teacher-Selected vs Learner-Selected Examples

- **Teacher-selected** examples:
 - Teacher chooses informative board states
 - Teacher supplies the correct move for each

Teacher-Selected vs Learner-Selected Examples

- **Teacher-selected** examples:
 - Teacher chooses informative board states
 - Teacher supplies the correct move for each
- **Learner-selected** examples with teacher:
 - Learner proposes board states it finds confusing

Teacher-Selected vs Learner-Selected Examples

- **Teacher-selected** examples:
 - Teacher chooses informative board states
 - Teacher supplies the correct move for each
- **Learner-selected** examples with teacher:
 - Learner proposes board states it finds confusing
 - Learner asks the teacher: "What is the correct move?"

Teacher-Selected vs Learner-Selected Examples

- **Teacher-selected** examples:
 - Teacher chooses informative board states
 - Teacher supplies the correct move for each
- **Learner-selected** examples with teacher:
 - Learner proposes board states it finds confusing
 - Learner asks the teacher: "What is the correct move?"
- **Self-play (no teacher):**
 - Learner controls both board states and training signals

Teacher-Selected vs Learner-Selected Examples

- **Teacher-selected** examples:
 - Teacher chooses informative board states
 - Teacher supplies the correct move for each
- **Learner-selected** examples with teacher:
 - Learner proposes board states it finds confusing
 - Learner asks the teacher: "What is the correct move?"
- **Self-play (no teacher):**
 - Learner controls both board states and training signals
 - It learns by playing **against itself**

Exploration vs Refinement in Self-Play

- In self-play, the learner can:
 - **Experiment** with novel board states not seen before

Exploration vs Refinement in Self-Play

- In self-play, the learner can:
 - **Experiment** with novel board states not seen before
 - **Refine** skill by playing variations of promising lines

Exploration vs Refinement in Self-Play

- In self-play, the learner can:
 - **Experiment** with novel board states not seen before
 - **Refine** skill by playing variations of promising lines
- Later chapters consider:
 - Random training examples from outside the learner's control

Exploration vs Refinement in Self-Play

- In self-play, the learner can:
 - **Experiment** with novel board states not seen before
 - **Refine** skill by playing variations of promising lines
- Later chapters consider:
 - Random training examples from outside the learner's control
 - Settings where learner can pose queries to an expert teacher

Exploration vs Refinement in Self-Play

- In self-play, the learner can:
 - **Experiment** with novel board states not seen before
 - **Refine** skill by playing variations of promising lines
- Later chapters consider:
 - Random training examples from outside the learner's control
 - Settings where learner can pose queries to an expert teacher
 - Settings where learner explores its environment autonomously

Attribute 3: Match of Training and Test Distributions

- Third attribute:
 - How well does the **training distribution** match the **test distribution**?

Attribute 3: Match of Training and Test Distributions

- Third attribute:
 - How well does the **training distribution** match the **test distribution**?
- General principle:
 - Learning is most reliable when training examples
 - follow a distribution **similar** to that of future test examples

Attribute 3: Match of Training and Test Distributions

- Third attribute:
 - How well does the **training distribution** match the **test distribution**?
- General principle:
 - Learning is most reliable when training examples
 - follow a distribution **similar** to that of future test examples
- In our checkers example:
 - Performance P : % of games won in the **world tournament**

Attribute 3: Match of Training and Test Distributions

- Third attribute:
 - How well does the **training distribution** match the **test distribution**?
- General principle:
 - Learning is most reliable when training examples
 - follow a distribution **similar** to that of future test examples
- In our checkers example:
 - Performance P : % of games won in the **world tournament**
 - Training experience E : possibly games **only against itself**

Danger of Mismatched Distributions

- If the system trains only by playing against itself:
 - The training experience may **not** represent the situations
 - it will face in the world tournament

Danger of Mismatched Distributions

- If the system trains only by playing against itself:
 - The training experience may **not** represent the situations it will face in the world tournament
- Possible issues:
 - It may never encounter crucial board states

Danger of Mismatched Distributions

- If the system trains only by playing against itself:
 - The training experience may **not** represent the situations it will face in the world tournament
- Possible issues:
 - It may never encounter crucial board states
 - It may learn strategies that work only against its own style

Danger of Mismatched Distributions

- If the system trains only by playing against itself:
 - The training experience may **not** represent the situations it will face in the world tournament
- Possible issues:
 - It may never encounter crucial board states
 - It may learn strategies that work only against its own style
 - Human champions may use very different lines of play

Danger of Mismatched Distributions

- If the system trains only by playing against itself:
 - The training experience may **not** represent the situations it will face in the world tournament
- Possible issues:
 - It may never encounter crucial board states
 - It may learn strategies that work only against its own style
 - Human champions may use very different lines of play
- In practice:
 - We often must learn from a distribution **different** from the test distribution
 - (e.g., the world champion may not teach the program)

Theoretical Assumption vs Practical Reality

- Most current theory of machine learning assumes:
 - **Training distribution = Test distribution**

Theoretical Assumption vs Practical Reality

- Most current theory of machine learning assumes:
 - **Training distribution = Test distribution**
- This assumption is crucial for:
 - Deriving formal theoretical results

Theoretical Assumption vs Practical Reality

- Most current theory of machine learning assumes:
 - **Training distribution = Test distribution**
- This assumption is crucial for:
 - Deriving formal theoretical results
- However, in practice:
 - This assumption is **often violated**

Theoretical Assumption vs Practical Reality

- Most current theory of machine learning assumes:
 - **Training distribution = Test distribution**
- This assumption is crucial for:
 - Deriving formal theoretical results
- However, in practice:
 - This assumption is **often violated**
 - We must remain aware of such mismatches

Decision: Training via Self-Play

- To proceed with our design, we choose:
 - System will train by **playing games against itself**

Decision: Training via Self-Play

- To proceed with our design, we choose:
 - System will train by **playing games against itself**
- Advantages:
 - No external trainer (teacher) is required

Decision: Training via Self-Play

- To proceed with our design, we choose:
 - System will train by **playing games against itself**
- Advantages:
 - No external trainer (teacher) is required
 - System can generate as much training data as time permits

Decision: Training via Self-Play

- To proceed with our design, we choose:
 - System will train by **playing games against itself**
- Advantages:
 - No external trainer (teacher) is required
 - System can generate as much training data as time permits
- Now we have a **fully specified learning task**

Checkers Learning Problem (Fully Specified)

- **Task T :** Playing checkers

Checkers Learning Problem (Fully Specified)

- **Task T :** Playing checkers
- **Performance measure P :**
 - Percent of games won in the **world tournament**

Checkers Learning Problem (Fully Specified)

- **Task T :** Playing checkers
- **Performance measure P :**
 - Percent of games won in the **world tournament**
- **Training experience E :**
 - Games played against itself (self-play)

What Still Remains to Design?

- Even after fixing T , P , and E , we must still choose:

What Still Remains to Design?

- Even after fixing T , P , and E , we must still choose:
 - ① The exact type of **knowledge to be learned**

What Still Remains to Design?

- Even after fixing T , P , and E , we must still choose:
 - ① The exact type of **knowledge to be learned**
 - ② A **representation** for this target knowledge

What Still Remains to Design?

- Even after fixing T , P , and E , we must still choose:
 - ① The exact type of **knowledge to be learned**
 - ② A **representation** for this target knowledge
 - ③ A suitable **learning mechanism** (algorithm)

What Still Remains to Design?

- Even after fixing T , P , and E , we must still choose:
 - ① The exact type of **knowledge to be learned**
 - ② A **representation** for this target knowledge
 - ③ A suitable **learning mechanism** (algorithm)
- These choices determine:
 - The internal structure of the learning system
 - How it generalizes from its training experience

1.2.2 Choosing the Target Function

- Next design choice:
 - **What type of knowledge** will be learned?

1.2.2 Choosing the Target Function

- Next design choice:
 - **What type of knowledge** will be learned?
 - How will this learned knowledge be **used by the performance program**?

1.2.2 Choosing the Target Function

- Next design choice:
 - **What type of knowledge** will be learned?
 - How will this learned knowledge be **used by the performance program**?
- Start with a checkers program that can:
 - Generate all **legal moves** from any board state

1.2.2 Choosing the Target Function

- Next design choice:
 - **What type of knowledge** will be learned?
 - How will this learned knowledge be **used by the performance program**?
- Start with a checkers program that can:
 - Generate all **legal moves** from any board state
- Then the program needs only to:
 - Learn how to **choose the best move** from among these legal moves

Checkers as a Representative Learning Task

- This checkers task is representative of a large class of problems:
 - The legal moves defining a **large search space** are known in advance

Checkers as a Representative Learning Task

- This checkers task is representative of a large class of problems:
 - The legal moves defining a **large search space** are known in advance
 - But the **best search strategy** is not known

Checkers as a Representative Learning Task

- This checkers task is representative of a large class of problems:
 - The legal moves defining a **large search space** are known in advance
 - But the **best search strategy** is not known
- Many optimization problems share this pattern:
 - Scheduling tasks

Checkers as a Representative Learning Task

- This checkers task is representative of a large class of problems:
 - The legal moves defining a **large search space** are known in advance
 - But the **best search strategy** is not known
- Many optimization problems share this pattern:
 - Scheduling tasks
 - Controlling manufacturing processes

Checkers as a Representative Learning Task

- This checkers task is representative of a large class of problems:
 - The legal moves defining a **large search space** are known in advance
 - But the **best search strategy** is not known
- Many optimization problems share this pattern:
 - Scheduling tasks
 - Controlling manufacturing processes
- In such problems:
 - Available actions or steps are well understood

Checkers as a Representative Learning Task

- This checkers task is representative of a large class of problems:
 - The legal moves defining a **large search space** are known in advance
 - But the **best search strategy** is not known
- Many optimization problems share this pattern:
 - Scheduling tasks
 - Controlling manufacturing processes
- In such problems:
 - Available actions or steps are well understood
 - The challenge is to learn the **best way to sequence or select** them

ChooseMove as a Target Function

- Given this setting, a natural type of information to learn is:
 - A **function** (or program) that chooses the best move for each board state

ChooseMove as a Target Function

- Given this setting, a natural type of information to learn is:
 - A **function** (or program) that chooses the best move for each board state
- Let us call this target function **ChooseMove**:

-

$$\text{ChooseMove} : B \rightarrow M$$

ChooseMove as a Target Function

- Given this setting, a natural type of information to learn is:
 - A **function** (or program) that chooses the best move for each board state
- Let us call this target function **ChooseMove**:

-

$$\text{ChooseMove} : B \rightarrow M$$

- B : set of legal board states

ChooseMove as a Target Function

- Given this setting, a natural type of information to learn is:
 - A **function** (or program) that chooses the best move for each board state
- Let us call this target function **ChooseMove**:

-

$$\text{ChooseMove} : B \rightarrow M$$

- B : set of legal board states
- M : set of legal moves

ChooseMove as a Target Function

- Given this setting, a natural type of information to learn is:
 - A **function** (or program) that chooses the best move for each board state
- Let us call this target function **ChooseMove**:

$$\text{ChooseMove} : B \rightarrow M$$

- B : set of legal board states
 - M : set of legal moves
- General pattern in machine learning:
 - Reduce the problem of improving performance P on task T

ChooseMove as a Target Function

- Given this setting, a natural type of information to learn is:
 - A **function** (or program) that chooses the best move for each board state
- Let us call this target function **ChooseMove**:

$$\text{ChooseMove} : B \rightarrow M$$

- B : set of legal board states
 - M : set of legal moves
- General pattern in machine learning:
 - Reduce the problem of improving performance P on task T
 - to the problem of **learning a particular target function** such as ChooseMove

ChooseMove as a Target Function

- Given this setting, a natural type of information to learn is:
 - A **function** (or program) that chooses the best move for each board state
- Let us call this target function **ChooseMove**:
 - $$\text{ChooseMove} : B \rightarrow M$$
 - B : set of legal board states
 - M : set of legal moves
- General pattern in machine learning:
 - Reduce the problem of improving performance P on task T
 - to the problem of **learning a particular target function** such as ChooseMove
- Therefore:
 - The **choice of target function** is a key design decision

Why ChooseMove is Difficult to Learn

- Although ChooseMove is an **obvious** target function:
 - It turns out to be **very difficult to learn** in our setting

Why ChooseMove is Difficult to Learn

- Although ChooseMove is an **obvious** target function:
 - It turns out to be **very difficult to learn** in our setting
- Reason:
 - Our training experience provides only **indirect feedback**
 - (complete games and final outcomes, not labeled moves)

Why ChooseMove is Difficult to Learn

- Although ChooseMove is an **obvious** target function:
 - It turns out to be **very difficult to learn** in our setting
- Reason:
 - Our training experience provides only **indirect feedback**
 - (complete games and final outcomes, not labeled moves)
- With such indirect training information:
 - Directly learning a mapping from board states to best moves
 - is a hard learning problem

Why ChooseMove is Difficult to Learn

- Although ChooseMove is an **obvious** target function:
 - It turns out to be **very difficult to learn** in our setting
- Reason:
 - Our training experience provides only **indirect feedback**
 - (complete games and final outcomes, not labeled moves)
- With such indirect training information:
 - Directly learning a mapping from board states to best moves
 - is a hard learning problem
- This motivates considering a **different** target function
 - One that is **easier to learn** from the available experience

Alternative Target: Evaluation Function V

- Alternative target function:
 - An **evaluation function** that assigns a numerical score to any board state

Alternative Target: Evaluation Function V

- Alternative target function:
 - An **evaluation function** that assigns a numerical score to any board state
- Denote this function by V :

-

$$V : B \rightarrow \mathbb{R}$$

Alternative Target: Evaluation Function V

- Alternative target function:
 - An **evaluation function** that assigns a numerical score to any board state
- Denote this function by V :

-

$$V : B \rightarrow \mathbb{R}$$

- B : set of legal board states

Alternative Target: Evaluation Function V

- Alternative target function:
 - An **evaluation function** that assigns a numerical score to any board state
- Denote this function by V :

-

$$V : B \rightarrow \mathbb{R}$$

- B : set of legal board states
- $\mathbb{R}$: set of real numbers

Alternative Target: Evaluation Function V

- Alternative target function:
 - An **evaluation function** that assigns a numerical score to any board state
- Denote this function by V :

-

$$V : B \rightarrow \mathbb{R}$$

- B : set of legal board states
 - $\mathbb{R}$: set of real numbers
- Intended meaning:
 - V should assign **higher scores to better board states**

Alternative Target: Evaluation Function V

- Alternative target function:
 - An **evaluation function** that assigns a numerical score to any board state
- Denote this function by V :

-

$$V : B \rightarrow \mathbb{R}$$

- B : set of legal board states
 - $\mathbb{R}$: set of real numbers
- Intended meaning:
 - V should assign **higher scores to better board states**
- In this setting:
 - Learning V will turn out to be **easier** than learning ChooseMove directly

Using V to Choose Moves

- If the system can successfully learn V :
 - It can use V to **select the best move** from any position

Using V to Choose Moves

- If the system can successfully learn V :
 - It can use V to **select the best move** from any position
- Procedure to choose a move from current board b :
 - ① Generate the **successor board state** for every legal move from b

Using V to Choose Moves

- If the system can successfully learn V :
 - It can use V to **select the best move** from any position
- Procedure to choose a move from current board b :
 - ① Generate the **successor board state** for every legal move from b
 - ② Apply V to each successor board state

Using V to Choose Moves

- If the system can successfully learn V :
 - It can use V to **select the best move** from any position
- Procedure to choose a move from current board b :
 - ① Generate the **successor board state** for every legal move from b
 - ② Apply V to each successor board state
 - ③ Choose the move whose successor state has the **highest value of V**

Using V to Choose Moves

- If the system can successfully learn V :
 - It can use V to **select the best move** from any position
- Procedure to choose a move from current board b :
 - ① Generate the **successor board state** for every legal move from b
 - ② Apply V to each successor board state
 - ③ Choose the move whose successor state has the **highest value of V**
- Thus:
 - A good approximation of V automatically yields a good move selector

Defining the Ideal Target Function V

- Many evaluation functions can lead to **optimal play**
 - As long as they assign higher scores to better states

Defining the Ideal Target Function V

- Many evaluation functions can lead to **optimal play**
 - As long as they assign higher scores to better states
- Nevertheless, it is useful to:
 - Define one particular **ideal** target function V

Defining the Ideal Target Function V

- Many evaluation functions can lead to **optimal play**
 - As long as they assign higher scores to better states
- Nevertheless, it is useful to:
 - Define one particular **ideal** target function V
 - Doing so will make it easier to design a training algorithm

Defining the Ideal Target Function V

- Many evaluation functions can lead to **optimal play**
 - As long as they assign higher scores to better states
- Nevertheless, it is useful to:
 - Define one particular **ideal** target function V
 - Doing so will make it easier to design a training algorithm
- Let $b \in B$ be an arbitrary legal board state
 - We now give a **specific, recursive definition** of $V(b)$

Recursive Definition of $V(b)$ (Final States)

- Define $V(b)$ for final board states as:

Recursive Definition of $V(b)$ (Final States)

- Define $V(b)$ for final board states as:
 - ① If b is a final board state that is **won**, then

$$V(b) = 100$$

Recursive Definition of $V(b)$ (Final States)

- Define $V(b)$ for final board states as:

- ① If b is a final board state that is **won**, then

$$V(b) = 100$$

- ② If b is a final board state that is **lost**, then

$$V(b) = -100$$

Recursive Definition of $V(b)$ (Final States)

- Define $V(b)$ for final board states as:

- 1 If b is a final board state that is **won**, then

$$V(b) = 100$$

- 2 If b is a final board state that is **lost**, then

$$V(b) = -100$$

- 3 If b is a final board state that is **drawn**, then

$$V(b) = 0$$

Recursive Definition of $V(b)$ (Final States)

- Define $V(b)$ for final board states as:

- ① If b is a final board state that is **won**, then

$$V(b) = 100$$

- ② If b is a final board state that is **lost**, then

$$V(b) = -100$$

- ③ If b is a final board state that is **drawn**, then

$$V(b) = 0$$

- These cases cover all **terminal positions** in the game

Recursive Definition of $V(b)$ (Non-Final States)

- For any board state b that is **not** a final state:
 - Define $V(b)$ in terms of the **best final board state** reachable

Recursive Definition of $V(b)$ (Non-Final States)

- For any board state b that is **not** a final state:
 - Define $V(b)$ in terms of the **best final board state** reachable
- Specifically:
 - Let b' be the **best final board state** that can be achieved
 - starting from b and playing **optimally** until the end of the game
 - (assuming the opponent also plays **optimally**)

Recursive Definition of $V(b)$ (Non-Final States)

- For any board state b that is **not** a final state:
 - Define $V(b)$ in terms of the **best final board state** reachable
- Specifically:
 - Let b' be the **best final board state** that can be achieved
 - starting from b and playing **optimally** until the end of the game
 - (assuming the opponent also plays **optimally**)
- Then:

$$V(b) = V(b')$$

Recursive Definition of $V(b)$ (Non-Final States)

- For any board state b that is **not** a final state:
 - Define $V(b)$ in terms of the **best final board state** reachable
- Specifically:
 - Let b' be the **best final board state** that can be achieved
 - starting from b and playing **optimally** until the end of the game
 - (assuming the opponent also plays **optimally**)
- Then:

$$V(b) = V(b')$$

- Together with the previous slide, this:
 - Recursively defines $V(b)$ for **every** legal board state b

Nonoperational Definition of V

- This recursive definition fully specifies $V(b)$
 - But it is **not usable** by our checkers program in practice

Nonoperational Definition of V

- This recursive definition fully specifies $V(b)$
 - But it is **not usable** by our checkers program in practice
- Why not?
 - Except for final states (won, lost, drawn),
 - computing $V(b)$ requires:
 - Searching ahead for the **optimal line of play**
 - all the way to the **end of the game**

Nonoperational Definition of V

- This recursive definition fully specifies $V(b)$
 - But it is **not usable** by our checkers program in practice
- Why not?
 - Except for final states (won, lost, drawn),
 - computing $V(b)$ requires:
 - Searching ahead for the **optimal line of play**
 - all the way to the **end of the game**
- This is not computationally feasible for actual play
 - Therefore, this definition is called a **nonoperational definition** of V

Goal: Learn an Operational Description of V

- Our aim in learning is to:
 - Discover an **operational description** of V

Goal: Learn an Operational Description of V

- Our aim in learning is to:
 - Discover an **operational description** of V
- An operational description:
 - Can be used by the checkers program to **evaluate states**

Goal: Learn an Operational Description of V

- Our aim in learning is to:
 - Discover an **operational description** of V
- An operational description:
 - Can be used by the checkers program to **evaluate states**
 - and **select moves** within realistic time bounds

Goal: Learn an Operational Description of V

- Our aim in learning is to:
 - Discover an **operational description** of V
- An operational description:
 - Can be used by the checkers program to **evaluate states**
 - and **select moves** within realistic time bounds
- Thus we have:
 - Reduced the learning task to:
 - Discovering an operational form of the **ideal target function** V

Function Approximation

- In general, it may be very difficult to:
 - Learn an operational form of V **perfectly**

Function Approximation

- In general, it may be very difficult to:
 - Learn an operational form of V **perfectly**
- In practice:
 - We often expect learning algorithms to acquire only an **approximation** to V

Function Approximation

- In general, it may be very difficult to:
 - Learn an operational form of V **perfectly**
- In practice:
 - We often expect learning algorithms to acquire only an **approximation** to V
- For this reason:
 - The process of learning the target function is often called
 - **function approximation**

Distinguishing V from the Learned Function

- Let:
 - V denote the **ideal** target evaluation function

Distinguishing V from the Learned Function

- Let:
 - V denote the **ideal** target evaluation function
- Let:
 - $\hat{V}$ (or some other symbol) denote the function that is **actually learned**

Distinguishing V from the Learned Function

- Let:
 - V denote the **ideal** target evaluation function
- Let:
 - $\hat{V}$ (or some other symbol) denote the function that is **actually learned**
- We use different symbols to emphasize:
 - $\hat{V}$ is generally only an **approximation** to V

Distinguishing V from the Learned Function

- Let:
 - V denote the **ideal** target evaluation function
- Let:
 - $\hat{V}$ (or some other symbol) denote the function that is **actually learned**
- We use different symbols to emphasize:
 - $\hat{V}$ is generally only an **approximation** to V
- Learning goal in this context:
 - Find an operational $\hat{V}$ that approximates V
 - well enough to produce **strong checkers play**

1.2.3 Choosing a Representation for the Target Function

- We have specified the **ideal target function** V

1.2.3 Choosing a Representation for the Target Function

- We have specified the **ideal target function** V
- Next design choice:
 - Choose a **representation** that the learning program will use

1.2.3 Choosing a Representation for the Target Function

- We have specified the **ideal target function** V
- Next design choice:
 - Choose a **representation** that the learning program will use
 - to describe the function c that it will actually learn

1.2.3 Choosing a Representation for the Target Function

- We have specified the **ideal target function** V
- Next design choice:
 - Choose a **representation** that the learning program will use
 - to describe the function c that it will actually learn
- As with earlier design choices, we again have **many options**

Possible Representations for the Target Function

- We could allow the program to represent c using:

Possible Representations for the Target Function

- We could allow the program to represent c using:
 - A large **table** with a distinct entry specifying the value for each distinct board state

Possible Representations for the Target Function

- We could allow the program to represent c using:
 - A large **table** with a distinct entry specifying the value for each distinct board state
 - A **collection of rules** that match against features of the board state

Possible Representations for the Target Function

- We could allow the program to represent c using:
 - A large **table** with a distinct entry specifying the value for each distinct board state
 - A **collection of rules** that match against features of the board state
 - A **quadratic polynomial** function of predefined board features

Possible Representations for the Target Function

- We could allow the program to represent c using:
 - A large **table** with a distinct entry specifying the value for each distinct board state
 - A **collection of rules** that match against features of the board state
 - A **quadratic polynomial** function of predefined board features
 - An **artificial neural network**

Possible Representations for the Target Function

- We could allow the program to represent c using:
 - A large **table** with a distinct entry specifying the value for each distinct board state
 - A **collection of rules** that match against features of the board state
 - A **quadratic polynomial** function of predefined board features
 - An **artificial neural network**
- Each of these is a different **representation choice** for c

Crucial Tradeoff in Representation Choice

- In general, this choice of representation involves a crucial **tradeoff**:

Crucial Tradeoff in Representation Choice

- In general, this choice of representation involves a crucial **tradeoff**:
 - On one hand:
 - We wish to pick a **very expressive** representation

Crucial Tradeoff in Representation Choice

- In general, this choice of representation involves a crucial **tradeoff**:
 - On one hand:
 - We wish to pick a **very expressive** representation
 - to allow representing as close an approximation as possible to the ideal target function V

Crucial Tradeoff in Representation Choice

- In general, this choice of representation involves a crucial **tradeoff**:
 - On one hand:
 - We wish to pick a **very expressive** representation
 - to allow representing as close an approximation as possible to the ideal target function V
 - On the other hand:
 - The more **expressive** the representation,

Crucial Tradeoff in Representation Choice

- In general, this choice of representation involves a crucial **tradeoff**:
 - On one hand:
 - We wish to pick a **very expressive** representation
 - to allow representing as close an approximation as possible to the ideal target function V
 - On the other hand:
 - The more **expressive** the representation,
 - the more **training data** the program will require

Crucial Tradeoff in Representation Choice

- In general, this choice of representation involves a crucial **tradeoff**:
 - On one hand:
 - We wish to pick a **very expressive** representation
 - to allow representing as close an approximation as possible to the ideal target function V
 - On the other hand:
 - The more **expressive** the representation,
 - the more **training data** the program will require
 - in order to choose among the alternative hypotheses it can represent

Crucial Tradeoff in Representation Choice

- In general, this choice of representation involves a crucial **tradeoff**:
 - On one hand:
 - We wish to pick a **very expressive** representation
 - to allow representing as close an approximation as possible to the ideal target function V
 - On the other hand:
 - The more **expressive** the representation,
 - the more **training data** the program will require
 - in order to choose among the alternative hypotheses it can represent
- So we must balance:
 - **Approximation power** vs. **data requirements**

Choosing a Simple Linear Representation

- To keep the discussion brief, we choose a **simple representation**

Choosing a Simple Linear Representation

- To keep the discussion brief, we choose a **simple representation**
- For any given board state, the function c will be:
 - Calculated as a **linear combination** of certain board features

Choosing a Simple Linear Representation

- To keep the discussion brief, we choose a **simple representation**
- For any given board state, the function c will be:
 - Calculated as a **linear combination** of certain board features
- We now specify these board features $x_1, \dots, x_6$

Board Features Used in the Linear Representation

- The following board features are used:

Board Features Used in the Linear Representation

- The following board features are used:
 - x_1 : the number of **black pieces** on the board

Board Features Used in the Linear Representation

- The following board features are used:
 - x_1 : the number of **black pieces** on the board
 - x_2 : the number of **red pieces** on the board

Board Features Used in the Linear Representation

- The following board features are used:
 - x_1 : the number of **black pieces** on the board
 - x_2 : the number of **red pieces** on the board
 - x_3 : the number of **black kings** on the board

Board Features Used in the Linear Representation

- The following board features are used:
 - x_1 : the number of **black pieces** on the board
 - x_2 : the number of **red pieces** on the board
 - x_3 : the number of **black kings** on the board
 - x_4 : the number of **red kings** on the board

Board Features Used in the Linear Representation

- The following board features are used:
 - x_1 : the number of **black pieces** on the board
 - x_2 : the number of **red pieces** on the board
 - x_3 : the number of **black kings** on the board
 - x_4 : the number of **red kings** on the board
 - x_5 : the number of **black pieces threatened by red**
 - i.e., pieces that can be captured on red's next turn

Board Features Used in the Linear Representation

- The following board features are used:
 - x_1 : the number of **black pieces** on the board
 - x_2 : the number of **red pieces** on the board
 - x_3 : the number of **black kings** on the board
 - x_4 : the number of **red kings** on the board
 - x_5 : the number of **black pieces threatened by red**
 - i.e., pieces that can be captured on red's next turn
 - x_6 : the number of **red pieces threatened by black**

Linear Form of the Learned Function $c(b)$

- The learning program will represent $c(b)$ as a **linear function** of these features:

Linear Form of the Learned Function $c(b)$

- The learning program will represent $c(b)$ as a **linear function** of these features:

$$c(b) = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5x_5 + w_6x_6$$

Linear Form of the Learned Function $c(b)$

- The learning program will represent $c(b)$ as a **linear function** of these features:

$$c(b) = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5x_5 + w_6x_6$$

- Here:
 - $w_0, w_1, \dots, w_6$ are **numerical coefficients** (weights)

Linear Form of the Learned Function $c(b)$

- The learning program will represent $c(b)$ as a **linear function** of these features:

$$c(b) = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5x_5 + w_6x_6$$

- Here:
 - $w_0, w_1, \dots, w_6$ are **numerical coefficients** (weights)
 - These weights are to be chosen by the **learning algorithm**

Interpretation of the Weights

- Learned values for the weights w_1 through w_6 determine:
 - The **relative importance** of the various board features

Interpretation of the Weights

- Learned values for the weights w_1 through w_6 determine:
 - The **relative importance** of the various board features
 - in determining the value of the board

Interpretation of the Weights

- Learned values for the weights w_1 through w_6 determine:
 - The **relative importance** of the various board features
 - in determining the value of the board
- The weight w_0 :
 - Provides an **additive constant** to the board value

Interpretation of the Weights

- Learned values for the weights w_1 through w_6 determine:
 - The **relative importance** of the various board features
 - in determining the value of the board
- The weight w_0 :
 - Provides an **additive constant** to the board value
- Thus:
 - Learning c corresponds to learning suitable values for $w_0, w_1, \dots, w_6$

Partial Design of a Checkers Learning Program

- We can now summarize our design choices so far:

Partial Design of a Checkers Learning Program

- We can now summarize our design choices so far:
 - We have elaborated the original formulation of the learning problem by choosing:
 - A type of **training experience**

Partial Design of a Checkers Learning Program

- We can now summarize our design choices so far:
 - We have elaborated the original formulation of the learning problem by choosing:
 - A type of **training experience**
 - A **target function** to be learned

Partial Design of a Checkers Learning Program

- We can now summarize our design choices so far:
 - We have elaborated the original formulation of the learning problem by choosing:
 - A type of **training experience**
 - A **target function** to be learned
 - A **representation** for this target function

Partial Design of a Checkers Learning Program

- We can now summarize our design choices so far:
 - We have elaborated the original formulation of the learning problem by choosing:
 - A type of **training experience**
 - A **target function** to be learned
 - A **representation** for this target function
- Our elaborated learning task is:

Partial Design of a Checkers Learning Program

- We can now summarize our design choices so far:
 - We have elaborated the original formulation of the learning problem by choosing:
 - A type of **training experience**
 - A **target function** to be learned
 - A **representation** for this target function
- Our elaborated learning task is:
 - **Task T** : playing checkers

Partial Design of a Checkers Learning Program

- We can now summarize our design choices so far:
 - We have elaborated the original formulation of the learning problem by choosing:
 - A type of **training experience**
 - A **target function** to be learned
 - A **representation** for this target function
- Our elaborated learning task is:
 - **Task T** : playing checkers
 - **Performance measure P** : percent of games won in the world tournament

Partial Design of a Checkers Learning Program

- We can now summarize our design choices so far:
 - We have elaborated the original formulation of the learning problem by choosing:
 - A type of **training experience**
 - A **target function** to be learned
 - A **representation** for this target function
- Our elaborated learning task is:
 - **Task T** : playing checkers
 - **Performance measure P** : percent of games won in the world tournament
 - **Training experience E** : games played against itself

Partial Design of a Checkers Learning Program

- We can now summarize our design choices so far:
 - We have elaborated the original formulation of the learning problem by choosing:
 - A type of **training experience**
 - A **target function** to be learned
 - A **representation** for this target function
- Our elaborated learning task is:
 - **Task T** : playing checkers
 - **Performance measure P** : percent of games won in the world tournament
 - **Training experience E** : games played against itself
 - **Target function**: $V : \text{Board} \rightarrow \mathbb{R}$

Partial Design of a Checkers Learning Program

- We can now summarize our design choices so far:
 - We have elaborated the original formulation of the learning problem by choosing:
 - A type of **training experience**
 - A **target function** to be learned
 - A **representation** for this target function
- Our elaborated learning task is:
 - **Task T** : playing checkers
 - **Performance measure P** : percent of games won in the world tournament
 - **Training experience E** : games played against itself
 - **Target function**: $V : \text{Board} \rightarrow \mathbb{R}$
 - **Target function representation**:
 - Linear function of features $x_1, \dots, x_6$ with weights $w_0, \dots, w_6$

Net Effect of the Representation Choice

- The first three items above:
 - Task T , performance measure P , training experience E

Net Effect of the Representation Choice

- The first three items above:
 - Task T , performance measure P , training experience E
 - Correspond to the **specification of the learning task**

Net Effect of the Representation Choice

- The first three items above:
 - Task T , performance measure P , training experience E
 - Correspond to the **specification of the learning task**
- The final two items:
 - Target function V

Net Effect of the Representation Choice

- The first three items above:
 - Task T , performance measure P , training experience E
 - Correspond to the **specification of the learning task**
- The final two items:
 - Target function V
 - Target function representation (linear in $x_1, \dots, x_6$)

Net Effect of the Representation Choice

- The first three items above:
 - Task T , performance measure P , training experience E
 - Correspond to the **specification of the learning task**
- The final two items:
 - Target function V
 - Target function representation (linear in $x_1, \dots, x_6$)
 - Are **design choices** for implementing the learning program

Net Effect of the Representation Choice

- The first three items above:
 - Task T , performance measure P , training experience E
 - Correspond to the **specification of the learning task**
- The final two items:
 - Target function V
 - Target function representation (linear in $x_1, \dots, x_6$)
 - Are **design choices** for implementing the learning program
- Notice the net effect of this set of design choices:
 - It reduces the problem of **learning a checkers strategy**

Net Effect of the Representation Choice

- The first three items above:
 - Task T , performance measure P , training experience E
 - Correspond to the **specification of the learning task**
- The final two items:
 - Target function V
 - Target function representation (linear in $x_1, \dots, x_6$)
 - Are **design choices** for implementing the learning program
- Notice the net effect of this set of design choices:
 - It reduces the problem of **learning a checkers strategy**
 - to the problem of learning values for the coefficients

$$w_0, w_1, \dots, w_6$$

Net Effect of the Representation Choice

- The first three items above:
 - Task T , performance measure P , training experience E
 - Correspond to the **specification of the learning task**
- The final two items:
 - Target function V
 - Target function representation (linear in $x_1, \dots, x_6$)
 - Are **design choices** for implementing the learning program
- Notice the net effect of this set of design choices:
 - It reduces the problem of **learning a checkers strategy**
 - to the problem of learning values for the coefficients

$$w_0, w_1, \dots, w_6$$

- in the chosen target function representation

1.2.4 Choosing a Function Approximation Algorithm

- In order to learn the target function f (our approximation to V), we require:

1.2.4 Choosing a Function Approximation Algorithm

- In order to learn the target function f (our approximation to V), we require:
 - A set of **training examples**
 - Each describes a specific board state b
 - And a corresponding **training value** $V_{\text{train}}(b)$

1.2.4 Choosing a Function Approximation Algorithm

- In order to learn the target function f (our approximation to V), we require:
 - A set of **training examples**
 - Each describes a specific board state b
 - And a corresponding **training value** $V_{\text{train}}(b)$
- Each training example is an ordered pair:

$$(b, V_{\text{train}}(b))$$

Example Training Instance

- Consider a training example describing a board state b in which:

Example Training Instance

- Consider a training example describing a board state b in which:
 - **Black has won** the game

Example Training Instance

- Consider a training example describing a board state b in which:
 - **Black has won** the game
 - For instance, $x_2 = 0$ indicates that red has no remaining pieces

Example Training Instance

- Consider a training example describing a board state b in which:
 - **Black has won** the game
 - For instance, $x_2 = 0$ indicates that red has no remaining pieces
- For such a final, winning board state:

$$V_{\text{train}}(b) = +100$$

Example Training Instance

- Consider a training example describing a board state b in which:
 - **Black has won** the game
 - For instance, $x_2 = 0$ indicates that red has no remaining pieces
- For such a final, winning board state:

$$V_{\text{train}}(b) = +100$$

- Below, we describe a procedure that:
 - First derives training examples from the **indirect training experience** available to the learner

Example Training Instance

- Consider a training example describing a board state b in which:
 - **Black has won** the game
 - For instance, $x_2 = 0$ indicates that red has no remaining pieces
- For such a final, winning board state:

$$V_{\text{train}}(b) = +100$$

- Below, we describe a procedure that:
 - First derives training examples from the **indirect training experience** available to the learner
 - Then **adjusts the weights** w_i to best fit these training examples

1.2.4.1 Estimating Training Values

- Recall our formulation of the learning problem:
 - The only training information available is
 - whether the game was **eventually won or lost**

1.2.4.1 Estimating Training Values

- Recall our formulation of the learning problem:
 - The only training information available is
 - whether the game was **eventually won or lost**
- However, we require training examples that:
 - Assign specific **scores to specific board states**

1.2.4.1 Estimating Training Values

- Recall our formulation of the learning problem:
 - The only training information available is
 - whether the game was **eventually won or lost**
- However, we require training examples that:
 - Assign specific **scores to specific board states**
- For **end-of-game** board states:
 - It is easy to assign a training value
 - (e.g., +100 for win, -100 for loss, 0 for draw)

Intermediate Board States

- It is much less obvious how to assign training values to:
 - The more numerous **intermediate** board states
 - that occur **before** the end of the game

Intermediate Board States

- It is much less obvious how to assign training values to:
 - The more numerous **intermediate** board states
 - that occur **before** the end of the game
- The fact that the game was eventually won or lost:
 - Does **not** necessarily indicate that every state along the game path
 - was good or bad

Intermediate Board States

- It is much less obvious how to assign training values to:
 - The more numerous **intermediate** board states
 - that occur **before** the end of the game
- The fact that the game was eventually won or lost:
 - Does **not** necessarily indicate that every state along the game path
 - was good or bad
- For example:
 - Even if the program **loses** the game,

Intermediate Board States

- It is much less obvious how to assign training values to:
 - The more numerous **intermediate** board states
 - that occur **before** the end of the game
- The fact that the game was eventually won or lost:
 - Does **not** necessarily indicate that every state along the game path
 - was good or bad
- For example:
 - Even if the program **loses** the game,
 - Some board states occurring **early** in the game may still deserve a **high** rating

Intermediate Board States

- It is much less obvious how to assign training values to:
 - The more numerous **intermediate** board states
 - that occur **before** the end of the game
- The fact that the game was eventually won or lost:
 - Does **not** necessarily indicate that every state along the game path
 - was good or bad
- For example:
 - Even if the program **loses** the game,
 - Some board states occurring **early** in the game may still deserve a **high** rating
 - The cause of the loss may be a **subsequent poor move**

A Simple Bootstrapping Approach

- Despite this ambiguity, a simple approach has been found to be:
 - **Surprisingly successful** in practice

A Simple Bootstrapping Approach

- Despite this ambiguity, a simple approach has been found to be:
 - **Surprisingly successful** in practice
- For any intermediate board state b , assign:

$$V_{\text{train}}(b) \leftarrow f(\text{Successor}(b))$$

where:

A Simple Bootstrapping Approach

- Despite this ambiguity, a simple approach has been found to be:
 - **Surprisingly successful** in practice
- For any intermediate board state b , assign:

$$V_{\text{train}}(b) \leftarrow f(\text{Successor}(b))$$

where:

- f is the learner's current approximation to V

A Simple Bootstrapping Approach

- Despite this ambiguity, a simple approach has been found to be:
 - **Surprisingly successful** in practice
- For any intermediate board state b , assign:

$$V_{\text{train}}(b) \leftarrow f(\text{Successor}(b))$$

where:

- f is the learner's current approximation to V
- $\text{Successor}(b)$ is the next board state after b
- for which it is again the **program's turn** to move
- (i.e., after the program's move and the opponent's response)

A Simple Bootstrapping Approach

- Despite this ambiguity, a simple approach has been found to be:
 - **Surprisingly successful** in practice
- For any intermediate board state b , assign:

$$V_{\text{train}}(b) \leftarrow f(\text{Successor}(b))$$

where:

- f is the learner's current approximation to V
- $\text{Successor}(b)$ is the next board state after b
- for which it is again the **program's turn** to move
- (i.e., after the program's move and the opponent's response)
- Summary rule for estimating training values:

$$V_{\text{train}}(b) \leftarrow f(\text{Successor}(b))$$

Numeric Example of Bootstrapping with Successor States

- Consider a single game trajectory:

$$b_1 \rightarrow b_2 \rightarrow b_3 \rightarrow b_4$$

where b_4 is a terminal **winning** state.

Numeric Example of Bootstrapping with Successor States

- Consider a single game trajectory:

$$b_1 \rightarrow b_2 \rightarrow b_3 \rightarrow b_4$$

where b_4 is a terminal **winning** state.

- True terminal value:

$$V_{\text{train}}(b_4) = 100$$

Numeric Example of Bootstrapping with Successor States

- Consider a single game trajectory:

$$b_1 \rightarrow b_2 \rightarrow b_3 \rightarrow b_4$$

where b_4 is a terminal **winning** state.

- True terminal value:

$$V_{\text{train}}(b_4) = 100$$

- Initialize estimates:

$$f(b_1) = f(b_2) = f(b_3) = f(b_4) = 0, \quad \eta = 0.5$$

Numeric Example of Bootstrapping with Successor States

- Consider a single game trajectory:

$$b_1 \rightarrow b_2 \rightarrow b_3 \rightarrow b_4$$

where b_4 is a terminal **winning** state.

- True terminal value:

$$V_{\text{train}}(b_4) = 100$$

- Initialize estimates:

$$f(b_1) = f(b_2) = f(b_3) = f(b_4) = 0, \quad \eta = 0.5$$

Step 1: Update b_4 (terminal state)

$$f(b_4) \leftarrow 0 + 0.5(100 - 0) = 50$$

Numeric Example of Bootstrapping with Successor States

- Consider a single game trajectory:

$$b_1 \rightarrow b_2 \rightarrow b_3 \rightarrow b_4$$

where b_4 is a terminal **winning** state.

- True terminal value:

$$V_{\text{train}}(b_4) = 100$$

- Initialize estimates:

$$f(b_1) = f(b_2) = f(b_3) = f(b_4) = 0, \quad \eta = 0.5$$

Step 1: Update b_4 (terminal state)

$$f(b_4) \leftarrow 0 + 0.5(100 - 0) = 50$$

Step 2: Update b_3 using $\text{Successor}(b_3) = b_4$

$$V_{\text{train}}(b_3) = f(b_4) = 50$$

Numeric Example (continued)

Recall after previous updates:

$$f(b_4) = 50, \quad f(b_3) = 25, \quad f(b_2) = 0, \quad f(b_1) = 0$$

Numeric Example (continued)

Recall after previous updates:

$$f(b_4) = 50, \quad f(b_3) = 25, \quad f(b_2) = 0, \quad f(b_1) = 0$$

Step 3: Update b_2 using $\text{Successor}(b_2) = b_3$

$$V_{\text{train}}(b_2) = f(b_3) = 25$$

$$f(b_2) \leftarrow 0 + 0.5(25 - 0) = 12.5$$

Numeric Example (continued)

Recall after previous updates:

$$f(b_4) = 50, \quad f(b_3) = 25, \quad f(b_2) = 0, \quad f(b_1) = 0$$

Step 3: Update b_2 using $\text{Successor}(b_2) = b_3$

$$V_{\text{train}}(b_2) = f(b_3) = 25$$

$$f(b_2) \leftarrow 0 + 0.5(25 - 0) = 12.5$$

Step 4: Update b_1 using $\text{Successor}(b_1) = b_2$

$$V_{\text{train}}(b_1) = f(b_2) = 12.5$$

$$f(b_1) \leftarrow 0 + 0.5(12.5 - 0) = 6.25$$

Numeric Example (continued)

Recall after previous updates:

$$f(b_4) = 50, \quad f(b_3) = 25, \quad f(b_2) = 0, \quad f(b_1) = 0$$

Step 3: Update b_2 using $\text{Successor}(b_2) = b_3$

$$V_{\text{train}}(b_2) = f(b_3) = 25$$

$$f(b_2) \leftarrow 0 + 0.5(25 - 0) = 12.5$$

Step 4: Update b_1 using $\text{Successor}(b_1) = b_2$

$$V_{\text{train}}(b_1) = f(b_2) = 12.5$$

$$f(b_1) \leftarrow 0 + 0.5(12.5 - 0) = 6.25$$

Result after one sweep:

$$f(b_1) = 6.25, \quad f(b_2) = 12.5, \quad f(b_3) = 25, \quad f(b_4) = 50$$

Values closer to the terminal state are larger, and value information has **propagated**

Why Use f to Define Its Own Training Targets?

- It may seem strange to:
 - Use the current version of f to estimate training values

Why Use f to Define Its Own Training Targets?

- It may seem strange to:
 - Use the current version of f to estimate training values
 - that will then be used to **refine this very same function**

Why Use f to Define Its Own Training Targets?

- It may seem strange to:
 - Use the current version of f to estimate training values
 - that will then be used to **refine this very same function**
- But notice:
 - We are using estimates of the value of $\text{Successor}(b)$

Why Use f to Define Its Own Training Targets?

- It may seem strange to:
 - Use the current version of f to estimate training values
 - that will then be used to **refine this very same function**
- But notice:
 - We are using estimates of the value of $\text{Successor}(b)$
 - to estimate the value of the **preceding** board state b

Why Use f to Define Its Own Training Targets?

- It may seem strange to:
 - Use the current version of f to estimate training values
 - that will then be used to **refine this very same function**
- But notice:
 - We are using estimates of the value of $\text{Successor}(b)$
 - to estimate the value of the **preceding** board state b
- Intuitively, this can make sense if:
 - f tends to be more **accurate** for board states

Why Use f to Define Its Own Training Targets?

- It may seem strange to:
 - Use the current version of f to estimate training values
 - that will then be used to **refine this very same function**
- But notice:
 - We are using estimates of the value of $\text{Successor}(b)$
 - to estimate the value of the **preceding** board state b
- Intuitively, this can make sense if:
 - f tends to be more **accurate** for board states
 - that are closer to the **end of the game**

Why Use f to Define Its Own Training Targets?

- It may seem strange to:
 - Use the current version of f to estimate training values
 - that will then be used to **refine this very same function**
- But notice:
 - We are using estimates of the value of $\text{Successor}(b)$
 - to estimate the value of the **preceding** board state b
- Intuitively, this can make sense if:
 - f tends to be more **accurate** for board states
 - that are closer to the **end of the game**
- In fact, under certain conditions (discussed in Chapter 13):
 - Iteratively estimating training values based on successor state values

Why Use f to Define Its Own Training Targets?

- It may seem strange to:
 - Use the current version of f to estimate training values
 - that will then be used to **refine this very same function**
- But notice:
 - We are using estimates of the value of $\text{Successor}(b)$
 - to estimate the value of the **preceding** board state b
- Intuitively, this can make sense if:
 - f tends to be more **accurate** for board states
 - that are closer to the **end of the game**
- In fact, under certain conditions (discussed in Chapter 13):
 - Iteratively estimating training values based on successor state values
 - can be proven to **converge** toward perfect estimates of V_{train}

1.2.4.2 Adjusting the Weights

- We now specify the learning algorithm for:
 - Choosing the weights w_i

1.2.4.2 Adjusting the Weights

- We now specify the learning algorithm for:
 - Choosing the weights w_i
 - to **best fit** the set of training examples $\{(b, V_{\text{train}}(b))\}$

1.2.4.2 Adjusting the Weights

- We now specify the learning algorithm for:
 - Choosing the weights w_i
 - to **best fit** the set of training examples $\{(b, V_{\text{train}}(b))\}$
- As a first step, we define what we mean by the **best fit**:
 - One common approach is to minimize the **squared error** E

1.2.4.2 Adjusting the Weights

- We now specify the learning algorithm for:
 - Choosing the weights w_i
 - to **best fit** the set of training examples $\{(b, V_{\text{train}}(b))\}$
- As a first step, we define what we mean by the **best fit**:
 - One common approach is to minimize the **squared error** E
 - between the training values $V_{\text{train}}(b)$

1.2.4.2 Adjusting the Weights

- We now specify the learning algorithm for:
 - Choosing the weights w_i
 - to **best fit** the set of training examples $\{(b, V_{\text{train}}(b))\}$
- As a first step, we define what we mean by the **best fit**:
 - One common approach is to minimize the **squared error** E
 - between the training values $V_{\text{train}}(b)$
 - and the values predicted by the hypothesis f

Minimizing Squared Error

- We seek the weights (equivalently, the function f) that:
 - Minimize the squared error E for the observed training examples

Minimizing Squared Error

- We seek the weights (equivalently, the function f) that:
 - Minimize the squared error E for the observed training examples
- In some settings (discussed in Chapter 6):
 - Minimizing the sum of squared errors is equivalent to

Minimizing Squared Error

- We seek the weights (equivalently, the function f) that:
 - Minimize the squared error E for the observed training examples
- In some settings (discussed in Chapter 6):
 - Minimizing the sum of squared errors is equivalent to
 - finding the **most probable hypothesis**

Minimizing Squared Error

- We seek the weights (equivalently, the function f) that:
 - Minimize the squared error E for the observed training examples
- In some settings (discussed in Chapter 6):
 - Minimizing the sum of squared errors is equivalent to
 - finding the **most probable hypothesis**
 - given the observed training data

Minimizing Squared Error

- We seek the weights (equivalently, the function f) that:
 - Minimize the squared error E for the observed training examples
- In some settings (discussed in Chapter 6):
 - Minimizing the sum of squared errors is equivalent to
 - finding the **most probable hypothesis**
 - given the observed training data
- Several algorithms are known for:
 - Finding weights of a linear function that minimize E

Requirements for Our Learning Algorithm

- In our case, we require an algorithm that:

Requirements for Our Learning Algorithm

- In our case, we require an algorithm that:
 - Will **incrementally refine** the weights as new training examples arrive

Requirements for Our Learning Algorithm

- In our case, we require an algorithm that:
 - Will **incrementally refine** the weights as new training examples arrive
 - Will be **robust** to errors in the **estimated** training values $V_{\text{train}}(b)$

Requirements for Our Learning Algorithm

- In our case, we require an algorithm that:
 - Will **incrementally refine** the weights as new training examples arrive
 - Will be **robust** to errors in the **estimated** training values $V_{\text{train}}(b)$
- One such algorithm is the:
 - **Least Mean Squares (LMS)** training rule

Requirements for Our Learning Algorithm

- In our case, we require an algorithm that:
 - Will **incrementally refine** the weights as new training examples arrive
 - Will be **robust** to errors in the **estimated** training values $V_{\text{train}}(b)$
- One such algorithm is the:
 - **Least Mean Squares (LMS)** training rule
- As discussed in Chapter 4:
 - LMS can be viewed as performing a **stochastic gradient-descent** search

Requirements for Our Learning Algorithm

- In our case, we require an algorithm that:
 - Will **incrementally refine** the weights as new training examples arrive
 - Will be **robust** to errors in the **estimated** training values $V_{\text{train}}(b)$
- One such algorithm is the:
 - **Least Mean Squares (LMS)** training rule
- As discussed in Chapter 4:
 - LMS can be viewed as performing a **stochastic gradient-descent** search
 - through the space of possible hypotheses (weight values)

Requirements for Our Learning Algorithm

- In our case, we require an algorithm that:
 - Will **incrementally refine** the weights as new training examples arrive
 - Will be **robust** to errors in the **estimated** training values $V_{\text{train}}(b)$
- One such algorithm is the:
 - **Least Mean Squares (LMS)** training rule
- As discussed in Chapter 4:
 - LMS can be viewed as performing a **stochastic gradient-descent** search
 - through the space of possible hypotheses (weight values)
 - to minimize the squared error E

LMS Weight Update Rule

- LMS weight update rule:

LMS Weight Update Rule

- LMS weight update rule:
 - ① For each training example $(b, V_{\text{train}}(b))$:
 - Use the current weights to calculate $f(b)$

LMS Weight Update Rule

- LMS weight update rule:
 - ① For each training example $(b, V_{\text{train}}(b))$:
 - Use the current weights to calculate $f(b)$
 - ② For each weight w_i , update it as:

$$w_i \leftarrow w_i + \eta(V_{\text{train}}(b) - f(b))x_i$$

LMS Weight Update Rule

- LMS weight update rule:
 - ① For each training example $(b, V_{\text{train}}(b))$:
 - Use the current weights to calculate $f(b)$
 - ② For each weight w_i , update it as:

$$w_i \leftarrow w_i + \eta(V_{\text{train}}(b) - f(b))x_i$$

- Here:
 - η is a small constant (e.g., 0.1)

LMS Weight Update Rule

- LMS weight update rule:
 - ① For each training example $(b, V_{\text{train}}(b))$:
 - Use the current weights to calculate $f(b)$
 - ② For each weight w_i , update it as:

$$w_i \leftarrow w_i + \eta(V_{\text{train}}(b) - f(b))x_i$$

- Here:
 - η is a small constant (e.g., 0.1)
 - that moderates the size of the weight update

Intuition Behind the LMS Update

- When the error $(V_{\text{train}}(b) - f(b))$ is **zero**:
 - No weights are changed

Intuition Behind the LMS Update

- When the error $(V_{\text{train}}(b) - f(b))$ is **zero**:
 - No weights are changed
- When $(V_{\text{train}}(b) - f(b))$ is **positive**:
 - $f(b)$ is **too low**

Intuition Behind the LMS Update

- When the error $(V_{\text{train}}(b) - f(b))$ is **zero**:
 - No weights are changed
- When $(V_{\text{train}}(b) - f(b))$ is **positive**:
 - $f(b)$ is **too low**
 - Each weight w_i is **increased** in proportion to the value of x_i

Intuition Behind the LMS Update

- When the error $(V_{\text{train}}(b) - f(b))$ is **zero**:
 - No weights are changed
- When $(V_{\text{train}}(b) - f(b))$ is **positive**:
 - $f(b)$ is **too low**
 - Each weight w_i is **increased** in proportion to the value of x_i
 - This **raises** the value of $f(b)$, reducing the error

Intuition Behind the LMS Update

- When the error $(V_{\text{train}}(b) - f(b))$ is **zero**:
 - No weights are changed
- When $(V_{\text{train}}(b) - f(b))$ is **positive**:
 - $f(b)$ is **too low**
 - Each weight w_i is **increased** in proportion to the value of x_i
 - This **raises** the value of $f(b)$, reducing the error
- When the value of some feature x_i is **zero**:
 - Its weight w_i is **not altered**, regardless of the error

Intuition Behind the LMS Update

- When the error $(V_{\text{train}}(b) - f(b))$ is **zero**:
 - No weights are changed
- When $(V_{\text{train}}(b) - f(b))$ is **positive**:
 - $f(b)$ is **too low**
 - Each weight w_i is **increased** in proportion to the value of x_i
 - This **raises** the value of $f(b)$, reducing the error
- When the value of some feature x_i is **zero**:
 - Its weight w_i is **not altered**, regardless of the error
 - So only weights corresponding to features that **occur** on the board are updated

Convergence Properties of LMS

- Surprisingly, in certain settings:
 - This simple weight-tuning method can be proven to **converge**

Convergence Properties of LMS

- Surprisingly, in certain settings:
 - This simple weight-tuning method can be proven to **converge**
 - to the **least squared error** approximation

Convergence Properties of LMS

- Surprisingly, in certain settings:
 - This simple weight-tuning method can be proven to **converge**
 - to the **least squared error** approximation
 - to the V_{train} values

Convergence Properties of LMS

- Surprisingly, in certain settings:
 - This simple weight-tuning method can be proven to **converge**
 - to the **least squared error** approximation
 - to the V_{train} values
- These convergence properties are discussed in more detail in Chapter 4

Convergence Properties of LMS

- Surprisingly, in certain settings:
 - This simple weight-tuning method can be proven to **converge**
 - to the **least squared error** approximation
 - to the V_{train} values
- These convergence properties are discussed in more detail in Chapter 4
- Thus, LMS provides:
 - A simple and effective algorithm for **adjusting the weights** w_i

Convergence Properties of LMS

- Surprisingly, in certain settings:
 - This simple weight-tuning method can be proven to **converge**
 - to the **least squared error** approximation
 - to the V_{train} values
- These convergence properties are discussed in more detail in Chapter 4
- Thus, LMS provides:
 - A simple and effective algorithm for **adjusting the weights** w_i
 - to approximate the desired evaluation function

1.2.5 The Final Design

- The final design of our checkers learning system can be described by **four distinct program modules**

1.2.5 The Final Design

- The final design of our checkers learning system can be described by **four distinct program modules**
- These four modules represent central components in many learning systems:

1.2.5 The Final Design

- The final design of our checkers learning system can be described by **four distinct program modules**
- These four modules represent central components in many learning systems:
 - ① Performance System

1.2.5 The Final Design

- The final design of our checkers learning system can be described by **four distinct program modules**
- These four modules represent central components in many learning systems:
 - ① Performance System
 - ② Critic

1.2.5 The Final Design

- The final design of our checkers learning system can be described by **four distinct program modules**
- These four modules represent central components in many learning systems:
 - ① Performance System
 - ② Critic
 - ③ Generalizer

1.2.5 The Final Design

- The final design of our checkers learning system can be described by **four distinct program modules**
- These four modules represent central components in many learning systems:
 - ① Performance System
 - ② Critic
 - ③ Generalizer
 - ④ Experiment Generator

1.2.5 The Final Design

- The final design of our checkers learning system can be described by **four distinct program modules**
- These four modules represent central components in many learning systems:
 - ① Performance System
 - ② Critic
 - ③ Generalizer
 - ④ Experiment Generator
- They are summarized in the conceptual diagram (Figure 1.1 in the book)

The Performance System

- The **Performance System** is the module that must solve the given **performance task**
 - In this case: **playing checkers**

The Performance System

- The **Performance System** is the module that must solve the given **performance task**
 - In this case: **playing checkers**
- It uses the **learned target function(s)** to:
 - Take an instance of a new problem as input
 - (here: a new initial game board)

The Performance System

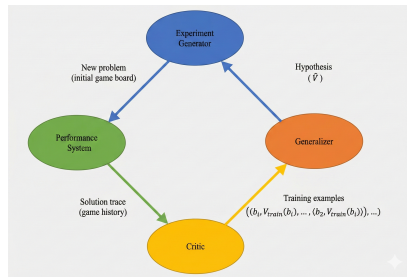
- The **Performance System** is the module that must solve the given **performance task**
 - In this case: **playing checkers**
- It uses the **learned target function(s)** to:
 - Take an instance of a new problem as input
 - (here: a new initial game board)
 - And produce a **trace of its solution** as output
 - (here: the full **game history**)

The Performance System

- The **Performance System** is the module that must solve the given **performance task**
 - In this case: **playing checkers**
- It uses the **learned target function(s)** to:
 - Take an instance of a new problem as input
 - (here: a new initial game board)
 - And produce a **trace of its solution** as output
 - (here: the full **game history**)
- In our checkers example:
 - The strategy used by the Performance System to select its next move
 - is determined by the learned evaluation function f

The Performance System

- The **Performance System** is the module that must solve the given **performance task**
 - In this case: **playing checkers**
- It uses the **learned target function(s)** to:
 - Take an instance of a new problem as input
 - (here: a new initial game board)
 - And produce a **trace of its solution** as output
 - (here: the full **game history**)
- In our checkers example:
 - The strategy used by the Performance System to select its next move
 - is determined by the learned evaluation function f
 - Therefore, we expect its performance to **improve** as f becomes increasingly accurate



The Critic

- The **Critic**:
 - Takes as input the **history or trace** of the game

The Critic

- The **Critic**:
 - Takes as input the **history or trace** of the game
 - Produces as output a set of **training examples** of the target function

The Critic

- The **Critic**:
 - Takes as input the **history or trace** of the game
 - Produces as output a set of **training examples** of the target function
- Each training example:
 - Corresponds to some game state b in the trace

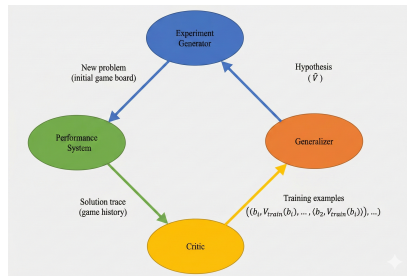
The Critic

- The **Critic**:
 - Takes as input the **history or trace** of the game
 - Produces as output a set of **training examples** of the target function
- Each training example:
 - Corresponds to some game state b in the trace
 - Along with an estimate $V_{\text{train}}(b)$ of the target function value

- The **Critic**:
 - Takes as input the **history or trace** of the game
 - Produces as output a set of **training examples** of the target function
- Each training example:
 - Corresponds to some game state b in the trace
 - Along with an estimate $V_{\text{train}}(b)$ of the target function value
- In our example:
 - The Critic corresponds to the training rule given by Equation (1.1)

The Critic

- The **Critic**:
 - Takes as input the **history or trace** of the game
 - Produces as output a set of **training examples** of the target function
- Each training example:
 - Corresponds to some game state b in the trace
 - Along with an estimate $V_{\text{train}}(b)$ of the target function value
- In our example:
 - The Critic corresponds to the training rule given by Equation (1.1)
 - Which estimates $V_{\text{train}}(b)$ using the value of $\text{Successor}(b)$ under the current f



The Generalizer

- The **Generalizer**:
 - Takes as input the **training examples**

The Generalizer

- The **Generalizer**:
 - Takes as input the **training examples**
 - Produces as output a **hypothesis** that estimates the target function

The Generalizer

- The **Generalizer**:
 - Takes as input the **training examples**
 - Produces as output a **hypothesis** that estimates the target function
- It **generalizes** from specific examples by:
 - Hypothesizing a function that covers the examples

The Generalizer

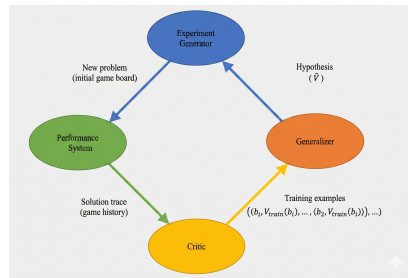
- The **Generalizer**:
 - Takes as input the **training examples**
 - Produces as output a **hypothesis** that estimates the target function
- It **generalizes** from specific examples by:
 - Hypothesizing a function that covers the examples
 - And also applies to other unseen cases

The Generalizer

- The **Generalizer**:
 - Takes as input the **training examples**
 - Produces as output a **hypothesis** that estimates the target function
- It **generalizes** from specific examples by:
 - Hypothesizing a function that covers the examples
 - And also applies to other unseen cases
- In our example:
 - The Generalizer corresponds to the **LMS algorithm**

The Generalizer

- The **Generalizer**:
 - Takes as input the **training examples**
 - Produces as output a **hypothesis** that estimates the target function
- It **generalizes** from specific examples by:
 - Hypothesizing a function that covers the examples
 - And also applies to other unseen cases
- In our example:
 - The Generalizer corresponds to the **LMS algorithm**
 - The output hypothesis is the function f described by the learned weights $w_0, \dots, w_6$



The Experiment Generator

- The **Experiment Generator**:
 - Takes as input the current hypothesis f

The Experiment Generator

- The **Experiment Generator**:
 - Takes as input the current hypothesis f
 - Outputs a new **problem** for the Performance System

The Experiment Generator

- The **Experiment Generator**:
 - Takes as input the current hypothesis f
 - Outputs a new **problem** for the Performance System
 - In this context: a new **initial board** for practice games

The Experiment Generator

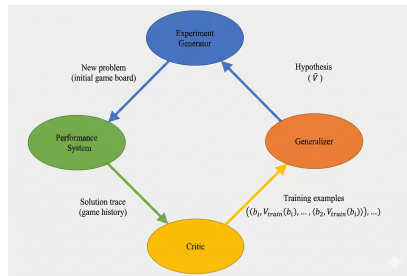
- The **Experiment Generator**:
 - Takes as input the current hypothesis f
 - Outputs a new **problem** for the Performance System
 - In this context: a new **initial board** for practice games
- Its role:
 - Pick new practice problems that **maximize learning rate**

The Experiment Generator

- The **Experiment Generator**:
 - Takes as input the current hypothesis f
 - Outputs a new **problem** for the Performance System
 - In this context: a new **initial board** for practice games
- Its role:
 - Pick new practice problems that **maximize learning rate**
- In our simple design:
 - The Experiment Generator always proposes the same starting board

The Experiment Generator

- The **Experiment Generator**:
 - Takes as input the current hypothesis f
 - Outputs a new **problem** for the Performance System
 - In this context: a new **initial board** for practice games
- Its role:
 - Pick new practice problems that **maximize learning rate**
- In our simple design:
 - The Experiment Generator always proposes the same starting board
 - More advanced versions could generate states to explore specific regions



A Generic Architecture for Learning Systems

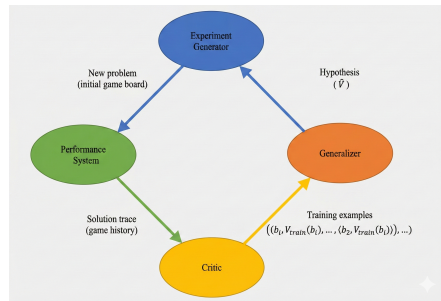
- Our design choices produce specific instantiations of:
 - Performance System
 - Critic
 - Generalizer
 - Experiment Generator

A Generic Architecture for Learning Systems

- Our design choices produce specific instantiations of:
 - Performance System
 - Critic
 - Generalizer
 - Experiment Generator
- Many machine learning systems can be usefully characterized in terms of these **four generic modules**

A Generic Architecture for Learning Systems

- Our design choices produce specific instantiations of:
 - Performance System
 - Critic
 - Generalizer
 - Experiment Generator
- Many machine learning systems can be usefully characterized in terms of these **four generic modules**
- They form a general conceptual architecture for:
 - Acting (Performance System)
 - Evaluating (Critic)
 - Learning/Generalizing (Generalizer)
 - Selecting new experiences (Experiment Generator)



figureGeneric ML System Architecture

Performance System in Deep Learning

- In deep learning, the **Performance System** corresponds to the **forward pass**.

Performance System in Deep Learning

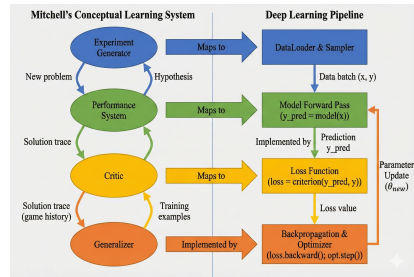
- In deep learning, the **Performance System** corresponds to the **forward pass**.
- The model uses its current parameters to produce:
 - predictions $\hat{y} = f_{\theta}(x)$

Performance System in Deep Learning

- In deep learning, the **Performance System** corresponds to the **forward pass**.
- The model uses its current parameters to produce:
 - predictions $\hat{y} = f_{\theta}(x)$
- This is analogous to:
 - The system “acting” using its learned knowledge
 - (Mitchell: playing a move in checkers)

Performance System in Deep Learning

- In deep learning, the **Performance System** corresponds to the **forward pass**.
- The model uses its current parameters to produce:
 - predictions $\hat{y} = f_{\theta}(x)$
- This is analogous to:
 - The system “acting” using its learned knowledge
 - (Mitchell: playing a move in checkers)
- As training progresses, the forward-pass performance improves as the learned function f_{θ} becomes more accurate.



The Critic in Deep Learning

- The **Critic** corresponds to the **loss function**.

The Critic in Deep Learning

- The **Critic** corresponds to the **loss function**.
- It evaluates how good or bad the model's prediction is:

$$\mathcal{L}(y, \hat{y})$$

The Critic in Deep Learning

- The **Critic** corresponds to the **loss function**.
- It evaluates how good or bad the model's prediction is:

$$\mathcal{L}(y, \hat{y})$$

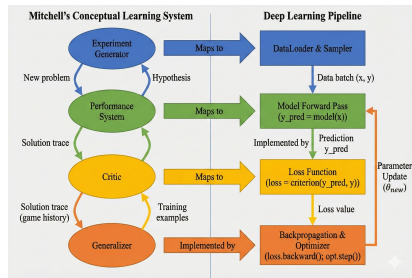
- This maps to Mitchell's idea of:
 - assigning credit or blame for each action
 - evaluating each state's value

The Critic in Deep Learning

- The **Critic** corresponds to the **loss function**.
- It evaluates how good or bad the model's prediction is:

$$\mathcal{L}(y, \hat{y})$$

- This maps to Mitchell's idea of:
 - assigning credit or blame for each action
 - evaluating each state's value
- In DL, the loss determines how much adjustment is needed.



The Generalizer in Deep Learning

- The **Generalizer** corresponds to:
 - **Backpropagation**
 - **Optimizer updates**

The Generalizer in Deep Learning

- The **Generalizer** corresponds to:
 - **Backpropagation**
 - **Optimizer updates**
- Weight updates:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

The Generalizer in Deep Learning

- The **Generalizer** corresponds to:
 - **Backpropagation**
 - **Optimizer updates**
- Weight updates:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

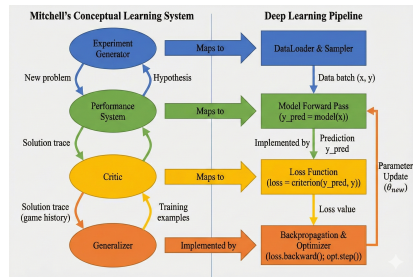
- This is identical in spirit to Mitchell's:
 - learning a new hypothesis from training data

The Generalizer in Deep Learning

- The **Generalizer** corresponds to:
 - **Backpropagation**
 - **Optimizer updates**
- Weight updates:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

- This is identical in spirit to Mitchell's:
 - learning a new hypothesis from training data
- The updated model $f_{\theta'}$ generalizes to new data.



Experiment Generator in Deep Learning

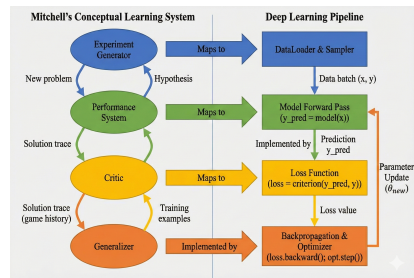
- In deep learning, the **Experiment Generator** corresponds to:
 - **DataLoader**
 - **Sampling strategy**
 - **Augmentations**

Experiment Generator in Deep Learning

- In deep learning, the **Experiment Generator** corresponds to:
 - **DataLoader**
 - **Sampling strategy**
 - **Augmentations**
- It determines:
 - What samples the model sees
 - In what order (random, curriculum)
 - With what variations (augmentations)

Experiment Generator in Deep Learning

- In deep learning, the **Experiment Generator** corresponds to:
 - **DataLoader**
 - **Sampling strategy**
 - **Augmentations**
- It determines:
 - What samples the model sees
 - In what order (random, curriculum)
 - With what variations (augmentations)
- This mirrors Mitchell's system:
 - selecting new “problems” to improve learning



Comparison: Original Learning System vs Deep Learning

- **Performance System**

- Checkers: Select next move
- Deep Learning: Forward pass (prediction)

Comparison: Original Learning System vs Deep Learning

- **Performance System**

- Checkers: Select next move
- Deep Learning: Forward pass (prediction)

- **Critic**

- Checkers: Evaluate game states
- Deep Learning: Loss function

Comparison: Original Learning System vs Deep Learning

- **Performance System**

- Checkers: Select next move
- Deep Learning: Forward pass (prediction)

- **Critic**

- Checkers: Evaluate game states
- Deep Learning: Loss function

- **Generalizer**

- Checkers: Hypothesis learner
- Deep Learning: Backprop + optimizer

Comparison: Original Learning System vs Deep Learning

- **Performance System**

- Checkers: Select next move
- Deep Learning: Forward pass (prediction)

- **Critic**

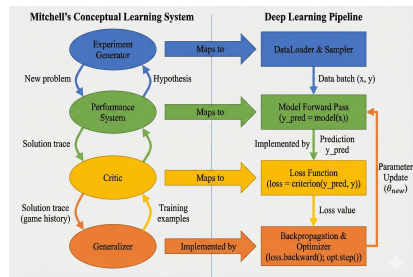
- Checkers: Evaluate game states
- Deep Learning: Loss function

- **Generalizer**

- Checkers: Hypothesis learner
- Deep Learning: Backprop + optimizer

- **Experiment Generator**

- Checkers: Produce new board states
- Deep Learning: DataLoader sampling



figureDeep Learning System Architecture

Summary of Design Choices (Figure 1.2)

- The sequence of design choices for the checkers program can be summarized as:

Summary of Design Choices (Figure 1.2)

- The sequence of design choices for the checkers program can be summarized as:
 - 1 **Determine type of training experience**

Summary of Design Choices (Figure 1.2)

- The sequence of design choices for the checkers program can be summarized as:
 - ① **Determine type of training experience**
 - ② **Determine target function**

Summary of Design Choices (Figure 1.2)

- The sequence of design choices for the checkers program can be summarized as:
 - 1 **Determine type of training experience**
 - 2 **Determine target function**
 - 3 **Determine representation** of the learned function

Summary of Design Choices (Figure 1.2)

- The sequence of design choices for the checkers program can be summarized as:
 - ① **Determine type of training experience**
 - ② **Determine target function**
 - ③ **Determine representation** of the learned function
 - ④ **Determine learning algorithm**

Summary of Design Choices (Figure 1.2)

- The sequence of design choices for the checkers program can be summarized as:
 - ① **Determine type of training experience**
 - ② **Determine target function**
 - ③ **Determine representation** of the learned function
 - ④ **Determine learning algorithm**
- These choices **constrain** the learning task in several ways

Constraints Imposed by the Design

- We have restricted the type of knowledge that can be acquired to:
 - A single **linear evaluation function**

Constraints Imposed by the Design

- We have restricted the type of knowledge that can be acquired to:
 - A single **linear evaluation function**
- We have further constrained this evaluation function to depend only on:
 - The six specific board features we selected

Constraints Imposed by the Design

- We have restricted the type of knowledge that can be acquired to:
 - A single **linear evaluation function**
 - We have further constrained this evaluation function to depend only on:
 - The six specific board features we selected
 - If the true target function V **can** be represented as:
 - A linear combination of these particular features
- then:
- Our program has a **good chance** to learn it

Constraints Imposed by the Design

- We have restricted the type of knowledge that can be acquired to:
 - A single **linear evaluation function**
- We have further constrained this evaluation function to depend only on:
 - The six specific board features we selected
- If the true target function V **can** be represented as:
 - A linear combination of these particular featuresthen:
 - Our program has a **good chance** to learn it
- If not:
 - The best we can hope for is a **good approximation**

Constraints Imposed by the Design

- We have restricted the type of knowledge that can be acquired to:
 - A single **linear evaluation function**
- We have further constrained this evaluation function to depend only on:
 - The six specific board features we selected
- If the true target function V **can** be represented as:
 - A linear combination of these particular featuresthen:
 - Our program has a **good chance** to learn it
- If not:
 - The best we can hope for is a **good approximation**
 - Because a program can never learn anything it **cannot represent**

Convergence and Theoretical Guarantees

- Suppose a good approximation to the true V can indeed be represented in the chosen form

Convergence and Theoretical Guarantees

- Suppose a good approximation to the true V can indeed be represented in the chosen form
- Question:
 - Is this learning technique guaranteed to find such an approximation?

Convergence and Theoretical Guarantees

- Suppose a good approximation to the true V can indeed be represented in the chosen form
- Question:
 - Is this learning technique guaranteed to find such an approximation?
- Chapter 13 provides a theoretical analysis showing that:
 - Under rather **restrictive assumptions**,

Convergence and Theoretical Guarantees

- Suppose a good approximation to the true V can indeed be represented in the chosen form
- Question:
 - Is this learning technique guaranteed to find such an approximation?
- Chapter 13 provides a theoretical analysis showing that:
 - Under rather **restrictive assumptions**,
 - Variations on this approach **do converge** to the desired evaluation function for certain types of search problems

Convergence and Theoretical Guarantees

- Suppose a good approximation to the true V can indeed be represented in the chosen form
- Question:
 - Is this learning technique guaranteed to find such an approximation?
- Chapter 13 provides a theoretical analysis showing that:
 - Under rather **restrictive assumptions**,
 - Variations on this approach **do converge** to the desired evaluation function for certain types of search problems
- Fortunately, practical experience suggests:
 - This approach to learning evaluation functions is often **successful**

Convergence and Theoretical Guarantees

- Suppose a good approximation to the true V can indeed be represented in the chosen form
- Question:
 - Is this learning technique guaranteed to find such an approximation?
- Chapter 13 provides a theoretical analysis showing that:
 - Under rather **restrictive assumptions**,
 - Variations on this approach **do converge** to the desired evaluation function for certain types of search problems
- Fortunately, practical experience suggests:
 - This approach to learning evaluation functions is often **successful**
 - Even outside the range of situations where such guarantees can be proven

How Good is This Particular Design?

- Would our program be able to learn well enough to:
 - Beat the human checkers world champion?

How Good is This Particular Design?

- Would our program be able to learn well enough to:
 - Beat the human checkers world champion?
- Probably **not**
 - In part, because the **linear function representation** for f is too simple

How Good is This Particular Design?

- Would our program be able to learn well enough to:
 - Beat the human checkers world champion?
- Probably **not**
 - In part, because the **linear function representation** for f is too simple
 - It may not capture the **nuances** of the game

How Good is This Particular Design?

- Would our program be able to learn well enough to:
 - Beat the human checkers world champion?
- Probably **not**
 - In part, because the **linear function representation** for f is too simple
 - It may not capture the **nuances** of the game
- However:
 - Given a more **sophisticated representation** of the target function,

How Good is This Particular Design?

- Would our program be able to learn well enough to:
 - Beat the human checkers world champion?
- Probably **not**
 - In part, because the **linear function representation** for f is too simple
 - It may not capture the **nuances** of the game
- However:
 - Given a more **sophisticated representation** of the target function,
 - This general approach can, in fact, be quite successful

Example: Tesauro's Backgammon Program

- Tesauro (1992, 1995) reports a similar design for a program that learns:
 - To play the game of **backgammon**

Example: Tesauro's Backgammon Program

- Tesauro (1992, 1995) reports a similar design for a program that learns:
 - To play the game of **backgammon**
- The program:
 - Learns an evaluation function over states of the game

Example: Tesauro's Backgammon Program

- Tesauro (1992, 1995) reports a similar design for a program that learns:
 - To play the game of **backgammon**
- The program:
 - Learns an evaluation function over states of the game
 - Represents this function using an **artificial neural network**

Example: Tesauro's Backgammon Program

- Tesauro (1992, 1995) reports a similar design for a program that learns:
 - To play the game of **backgammon**
- The program:
 - Learns an evaluation function over states of the game
 - Represents this function using an **artificial neural network**
 - Considers the **complete** description of the board state, rather than a small subset of features

Example: Tesauro's Backgammon Program

- Tesauro (1992, 1995) reports a similar design for a program that learns:
 - To play the game of **backgammon**
- The program:
 - Learns an evaluation function over states of the game
 - Represents this function using an **artificial neural network**
 - Considers the **complete** description of the board state, rather than a small subset of features
- After training on over **one million** self-generated training games:
 - The program was able to play very competitively

Example: Tesauro's Backgammon Program

- Tesauro (1992, 1995) reports a similar design for a program that learns:
 - To play the game of **backgammon**
- The program:
 - Learns an evaluation function over states of the game
 - Represents this function using an **artificial neural network**
 - Considers the **complete** description of the board state, rather than a small subset of features
- After training on over **one million** self-generated training games:
 - The program was able to play very competitively
 - With **top-ranked human** backgammon players

Alternative Algorithms for Checkers Learning

- Our design is only **one** of many possibilities

Alternative Algorithms for Checkers Learning

- Our design is only **one** of many possibilities
- Alternative approaches include:

Alternative Algorithms for Checkers Learning

- Our design is only **one** of many possibilities
- Alternative approaches include:
 - **Nearest neighbor** methods (Chapter 8):
 - Simply store given training examples

Alternative Algorithms for Checkers Learning

- Our design is only **one** of many possibilities
- Alternative approaches include:
 - **Nearest neighbor** methods (Chapter 8):
 - Simply store given training examples
 - For a new situation, find the **closest** stored situation and use its outcome

Alternative Algorithms for Checkers Learning

- Our design is only **one** of many possibilities
- Alternative approaches include:
 - **Nearest neighbor** methods (Chapter 8):
 - Simply store given training examples
 - For a new situation, find the **closest** stored situation and use its outcome
 - **Genetic algorithms** (Chapter 9):
 - Generate a large number of candidate checkers programs

Alternative Algorithms for Checkers Learning

- Our design is only **one** of many possibilities
- Alternative approaches include:
 - **Nearest neighbor** methods (Chapter 8):
 - Simply store given training examples
 - For a new situation, find the **closest** stored situation and use its outcome
 - **Genetic algorithms** (Chapter 9):
 - Generate a large number of candidate checkers programs
 - Allow them to play against each other

Alternative Algorithms for Checkers Learning

- Our design is only **one** of many possibilities
- Alternative approaches include:
 - **Nearest neighbor** methods (Chapter 8):
 - Simply store given training examples
 - For a new situation, find the **closest** stored situation and use its outcome
 - **Genetic algorithms** (Chapter 9):
 - Generate a large number of candidate checkers programs
 - Allow them to play against each other
 - Keep and further elaborate or mutate the most successful programs

Alternative Algorithms for Checkers Learning

- Our design is only **one** of many possibilities
- Alternative approaches include:
 - **Nearest neighbor** methods (Chapter 8):
 - Simply store given training examples
 - For a new situation, find the **closest** stored situation and use its outcome
 - **Genetic algorithms** (Chapter 9):
 - Generate a large number of candidate checkers programs
 - Allow them to play against each other
 - Keep and further elaborate or mutate the most successful programs
 - **Explanation-based learning** (Chapter 11):
 - Humans often analyze the reasons underlying specific successes and failures

Alternative Algorithms for Checkers Learning

- Our design is only **one** of many possibilities
- Alternative approaches include:
 - **Nearest neighbor** methods (Chapter 8):
 - Simply store given training examples
 - For a new situation, find the **closest** stored situation and use its outcome
 - **Genetic algorithms** (Chapter 9):
 - Generate a large number of candidate checkers programs
 - Allow them to play against each other
 - Keep and further elaborate or mutate the most successful programs
 - **Explanation-based learning** (Chapter 11):
 - Humans often analyze the reasons underlying specific successes and failures
 - They form strategic rules from these explanations

Closing Remarks on the Checkers Design

- The presented design:
 - Grounds our discussion of the **decisions** that must go into designing a learning method

Closing Remarks on the Checkers Design

- The presented design:
 - Grounds our discussion of the **decisions** that must go into designing a learning method
 - For a specific class of tasks such as learning to play checkers

Closing Remarks on the Checkers Design

- The presented design:
 - Grounds our discussion of the **decisions** that must go into designing a learning method
 - For a specific class of tasks such as learning to play checkers
- Key takeaway:
 - Every learning system must make **explicit choices** about training experience, target function, representation, and learning algorithm

Closing Remarks on the Checkers Design

- The presented design:
 - Grounds our discussion of the **decisions** that must go into designing a learning method
 - For a specific class of tasks such as learning to play checkers
- Key takeaway:
 - Every learning system must make **explicit choices** about training experience, target function, representation, and learning algorithm
 - Different choices lead to very different learning behaviors and capabilities